

1 Local repository, commits

Lokale map maken. Git-projecten zullen we opslaan in een lokale map, *niet* in de cloud (zoals OneDrive, Google Drive, Dropbox, ...). Git schrijft immers voortdurend bestanden weg en services als OneDrive zullen diezelfde bestanden tegelijk synchroniseren, dit kan conflicten, corrupte bestanden of prestatieproblemen veroorzaken.

- Open Windows Explorer.
- Typ in de adresbalk “C:\”.
- Klik links in de knoppenbalk op **Nieuw, Map**.
- Geef de map de naam “Projecten”.
- Maak nog een map aan in C:\Projecten met de naam “gittest”.

Lokale map openen in VSCode. Een project bestaat typisch niet uit één bestand, maar uit heel wat bestanden die worden verzameld in een map. We openen in VSCode de lokale map die net is gemaakt.

- Zorg ervoor dat er geen andere bestanden open staan in VSCode: **file, Close Folder**.
- Open de map C:\Projecten\gittest in VSCode via **File, Open Folder**.
- Maak meteen een nieuw tekstbestand aan (**ctrl+N**): **File, New Text File**. Typ hierin enkel je naam (lijn 1) en opleiding (lijn 2).
- Sla dit bestand op (**ctrl+S**): **File, Save** met als naam “gegevens.txt”.

Lokaal repository initialiseren. Op dit moment is de werkmap een gewone bestandsmap zonder versiecontrole. Om die te activeren, maken we er een Git-repository van.

- Klik in VSCode op het icoon **Source Control** (links in de knoppenbalk).
- Klik op **Manage Workspace Trust** en daarna op **Trust**. Code uit een onbekende bron open je veiligheidshalve in **Restricted mode**, zodat deze enkel kan worden gelezen en aangepast maar bijvoorbeeld niet uitgevoerd.
- Klik op **Initialize Repository** en **Yes** om de repository te initialiseren.
 - Je ziet dat het bestand **gegevens.txt** wordt vermeld onder **Changes**. Onderaan in de zijbalk is de **Graph** nog leeg, hier verschijnt later de inhoud van de repository.

Eerste commit. Nu kunnen we een het eerste bestand aan de repository doorvoeren.

- Tik in het vak **Message** onder **Changes** een beschrijving van de aanpassingen, in dit geval bijvoorbeeld “Add initial content”. Het icoontje rechts in het **Message**-vak laat toe een automatische beschrijving te genereren.
- Klik op **Commit**, en kies voor **Always** bij de pop-up vraag over **Changes**¹.
 - De `gegevens.txt` verschijnt in het **Graph**-overzicht, onder de beschrijving van de commit, met daarnaast de letter **A** (added file).

Aanpassingen doorvoeren aan een bestand. Nu zullen we het bestand `gegevens.txt` aanpassen en de aanpassingen doorvoeren naar de repository.

- Voeg aan het bestand `gegevens.txt` een derde lijn toe met je woonplaats. Sla het bestand op (**ctrl+S**).
- De bestandsnaam `gegevens.txt` is nu *drie* keer te zien in de **VSCode**-omgeving:
 - Het bestand in het editorvenster, met de laatste aanpassingen, is opgeslagen maar zit nog niet in de repository.
 - Klik op `gegevens.txt` onder **Changes**. Een nieuw venster opent in de editor met daarin de verschillen tussen de recentste versie van het bestand en de versie die in de repository staat. De toegevoegde lijn wordt groen gemarkeerd.
 - Klik op `gegevens.txt` onder **Graph**. Een nieuw venster opent in de editor met daarin de versie van het bestand zoals die is opgeslagen in de meest recente commit van de repository. Deze bevat nog maar twee lijnen.

Git bewaart dus zowel de huidige bestanden in je werkmapij als eerdere versies in de repository.

- Vul opnieuw een beschrijving van de aanpassingen aan in het vak **Message** onder **Changes**, bijvoorbeeld “Add location” en klik op **Commit**.
 - De **Graph** toont nu twee commits, met de meest recente bovenaan en telkens met het bestand `gegevens.txt`. Door er op te klikken kan je de versies vergelijken.
 - Naast de meest recente commit staat de letter **M** (modified file).

Met een commit leg je een versie van je project permanent vast in de repository, de message laat toe om later de juiste versie terug te vinden. De repository bevat dus alle versies van je project, ook al zijn bestanden aangepast of zelf verwijderd uit de werkmapij.

¹Hierdoor voegt **VSCode** de gewijzigde bestanden automatisch toe aan de commit. Later bekijken we hoe Git normaal een afzonderlijke “stage”-stap gebruikt.

Bestanden toevoegen en verwijderen. Nu zullen we nieuwe bestanden toevoegen en een bestaand bestand verwijderen.

- Maak twee nieuwe bestanden (**ctrl+N**) met als namen **br01.txt** (willekeurige inhoud) en **bestanden.txt** (bevat op elke lijn een bestandsnaam). Sla deze bestanden op (**ctrl+S**) en voer een commit uit.
 - In de **Graph** staan inmiddels drie commits, met daarbij telkens enkel de bestanden die in die commit zijn toegevoegd (**A**) of aangepast (**M**).
- Verwijder via **Explorer** het bestand **br01.txt** uit de map (selecteren + **Delete**).
- Schrap de bestandsnaam **br01.txt** uit de opsomming in **bestanden.txt** en commit de verwijderingen.
 - In **Source Control** wordt het bestand **br01.txt** gemarkeerd met de letter **D** (deleted file).
 - De geschrapte lijn staat rood gemarkeerd en zonder lijnnummer in het bestand **bestanden.txt** onder de recentste commit in de **Graph**.

Ook al is de informatie uit de map of uit een bestand verwijderd (zie **Explorer**), toch verdwijnt er geen informatie uit repository (zie **Source Control**).

Terminologie. Hiermee zijn alvast drie belangrijke begrippen geïntroduceerd:

Werkmap: de lokale map waarin de bestanden van het project staan.

Commit: een foto van de bestanden in de werkmap op één bepaald moment.

Repository: het geheel van alle commits en dus alle versies van de bestanden.

2 Repository publiceren op GitHub

Tot nu toe staat de repository alleen op de eigen computer. Dat is niet erg veilig, je kan informatie verliezen door een defect of diefstal van je laptop. Het is op die manier ook niet mogelijk om samen te werken met anderen. Daarom zullen we een kopie van de repository online bewaren, op GitHub.

GitHub-account aanmaken. Wie nog geen GitHub-account heeft, maakt er eerst één aan.

- Open een browser en ga naar <https://github.com>.
- Klik op **Sign Up**.
- Maak een account aan met je KU Leuven-adres². Heb je al een eigen GitHub-account, dan gebruik je die *niet* in de context van deze cursus.
- Meld aan op GitHub.

²Extra functionaliteit van KU Leuven-account?

Repository publiceren vanuit VSCode. De lokale repository `gittest` zullen we nu publiceren op GitHub.

-
- Klik in **Source Control** op **Publish Branch**. Deze knop staat er enkel als er geen hangende aanpassingen meer zijn die op een commit wachten.
- Kies **Publish to GitHub Private Repository**. Een private repository is enkel zichtbaar voor de eigenaar en voor personen die expliciet toegang krijgen. Een public repository is zichtbaar voor iedereen.
- Meld eventueel aan met je GitHub-account.
- Bevestig de publicatie.

Repository bekijken op GitHub. Na enkele seconden verschijnt de repository ook op GitHub.

- In de browser, klik op **Repositories** op je GitHub-pagina en daarna op de naam van de repository `gittest`.
- Verifieer dat de bestanden `gegevens.txt` en `bestanden.txt` zichtbaar zijn, met inhoud volgens de recentste commit.

Op dit moment bevat GitHub dus dezelfde commits als de lokale repository.

Nieuwe wijzigingen lokaal toevoegen. We zullen nu een nieuwe wijziging lokaal maken.

- Voeg aan `gegevens.txt` een vierde lijn toe met je favoriete kleur en sla het bestand op (**Ctrl+S**).
- Klik op **Commit**, verifieer de wijzigingen in **explorer** en vernieuw het bestand `gegevens.txt` op GitHub (**F5**).
 - Zoals verwacht is het lokale bestand aangepast.
 - De favoriete kleur is echter nog niet zichtbaar op GitHub.
 - In **Source Control** staat de laatste commit in een andere kleur dan de voorgaande commits. Twee icoontjes (concentrische cirkels en wolk) geven aan welke de actieve lokale commit is, en welke de recentste commit is die op GitHub staat.

De commit is lokaal opgeslagen, maar nog niet naar GitHub gestuurd.

Push uitvoeren. Om de nieuwe commit naar GitHub te sturen, gebruiken we een *push*.

- Klik in **Source Control** op **Sync Changes**.
- Vernieuw de GitHub-webpagina (**F5**) en verifieer dat de kleur nu wel zichtbaar is in `gegevens.txt`.

Wijzigingen rechtstreeks op GitHub aanbrengen. Niet alle wijzigingen moeten vanaf de eigen computer gebeuren. Een bestand kan ook rechtstreeks op GitHub worden aangepast.

- Open op GitHub het bestand `gegevens.txt` en klik op het potlood-icoon (`Edit this file`).
- Zet op de vijfde lijn je favoriete hobby.
- Klik op `Commit changes` en bevestig.
- Ga na of de hobby in `VSCoDe` zichtbaar is. Dat blijkt nog niet het geval te zijn.

Pull uitvoeren. GitHub bevat nu een recentere versie dan de lokale repository, om de wijzigingen van GitHub naar de eigen computer te halen, gebruiken we een *pull*.

- Klik op `Sync Changes`, het icoontje met de twee pijltjes helemaal links onderaan het `VSCoDe`-scherm³
- Verifieer dat de hobby nu ook lokaal zichtbaar is.

Terminologie.

- Lokale repository: staat op je eigen computer.
- Commit: wijzigingen vastleggen in de lokale repository.
- Remote repository: staat online, op GitHub.
- Push: lokale wijzigingen naar GitHub sturen.
- Pull: wijzigingen op GitHub naar je eigen computer halen.

3 Werken met branches

Tot nu toe hebben we telkens rechtstreeks gewerkt op de hoofdversie van de repository. Soms willen we echter veranderingen uitproberen zonder de bestaande versie te wijzigen. Denk aan een computerprogramma waarvan versie 1 correct werkt, maar waarvan je een deel wil herschrijven om het sneller te maken. Zolang je de nieuwe versie nog niet getest hebt, en geverifieerd dat deze effectief sneller is, wil je de bestaande versie niet overschrijven maar behouden en blijven gebruiken.

Daarvoor gebruiken we een *branch*. Een branch is een alternatieve versie van dezelfde repository. Elke branch heeft zijn eigen geschiedenis van commits.

³Verschil met Pull bij Graph?

De huidige branch bekijken.

- Helemaal onderaan links in **VSCoDe** verschijnt de naam van de huidige branch, op dit moment heet deze “**main**”. Dit is de naam van de hoofdversie van de repository. Tot nu toe werden alle commits op deze branch gemaakt.
- Verifieer dat dezelfde branchnaam op GitHub vermeld staat. De branch wordt er met hetzelfde symbool aangeduid als de **Source control**-knop in **VSCoDe**.

Een nieuwe branch maken. We zullen nu een nieuwe branch maken waarin we wijzigingen kunnen uitproberen.

- Klik in **VSCoDe** onderaan links op de branchnaam **main**.
- Kies in het lijstje opties dat verschijnt voor **Create New Branch**.
- Geef de branch de naam “**vakken**” en druk op **Enter**.
- Door opnieuw links onderaan op de branchnaam te klikken, kan je snel wisselen tussen **vakken** en **main**. Verifieer dat beide branches momenteel nog identiek zijn.
- Verifieer dat de nieuwe branch **vakken** momenteel enkel lokaal bestaat en nog niet te zien op GitHub.

Een wijziging maken op de nieuwe branch, local repository. We zullen nu een wijziging maken op de branch **vakken**.

- Selecteer de branch **vakken** en maak een nieuw bestand **vakken.txt**.
- Zet in het bestand een lijst van je favoriete vakken.
- Voer een **Commit** uit.
- Schakel tussen beide branches en verifieer dat de wijziging enkel zichtbaar is op de branch **vakken**.

Een nieuwe brach op de GitHub-repository. Nu maken we een nieuwe branch op GitHub aan.

TOT HIER: nu nieuwe branch op GitHub maken, en dan lokaal pushen en pullen. Ander voorbeeld dan Copilot-voorstel (geen conflicten, enkel verschillende bestanden)

-
- Schakel terug naar **main**.
- Open **gegevens.txt**.

- Voeg onderaan een nieuwe lijn toe met je favoriete vakantiebestemming.
- Sla het bestand op.
- Commit de wijziging met bijvoorbeeld `Add holiday destination`.

De verschillen bekijken. Vergelijk nu beide branches.

- Schakel naar `main`.
- Open `gegevens.txt`.

Deze versie bevat:

- de oorspronkelijke woonplaats;
- de vakantiebestemming.

Schakel vervolgens naar `woonplaats`.

- Open opnieuw `gegevens.txt`.

Deze versie bevat:

- de gewijzigde woonplaats;
- geen vakantiebestemming.

Beide branches bevatten dus verschillende versies van hetzelfde project.

Branches publiceren op GitHub. Momenteel bestaat de branch `woonplaats` alleen lokaal.

- Schakel naar de branch `woonplaats`.
- Open Source Control.
- Klik op `Publish Branch` of `Sync Changes`.

De branch wordt nu ook naar GitHub verzonden.

- Open de repository op GitHub.
- Klik boven de bestandenlijst op de branchselector.
- Controleer dat zowel `main` als `woonplaats` zichtbaar zijn.

Terminologie.

- **branch**: een alternatieve versie van de repository.
- **main**: de hoofdbbranch van de repository, waarin de stabiele versie van het project wordt bewaard.

Een typische workflow is om commits niet rechtstreeks op de hoofdbbranch te maken, maar eerst een nieuwe branch te maken, daar de wijzigingen uit te proberen en pas als alles correct werkt, de wijzigingen samen te voegen met de hoofdbbranch. Dat proces van samenvoegen heet *merge* en komt in de volgende sectie aan bod.

4 Samenvoegen van branches: merge

Ander voorbeeld dan Copilot-voorstel 1. geen conflicten, enkel bestaande verschillende bestanden 2. geen conflicten, maar wel verschillende aanpassingen in hetzelfde bestand 3. conflicten, waarbij expliciet gekozen moet worden welke versie behouden blijft

Pas hier branches introduceren.

Nieuwe terminologie branch main merge

Meer niet.

Geen:

rebase cherry pick detached HEAD Verhaal

”Je wil iets uitproberen zonder de hoofdversie stuk te maken.”

Praktische oefening

Maak branch:

woonplaats

Wijzig:

Gent

naar

Kortrijk

Commit.

Schakel terug naar main.

Laat studenten kijken naar gegevens.txt.

De wijziging is weg.

Schakel opnieuw naar woonplaats.

De wijziging is terug.

Dit moment werkt altijd goed didactisch.

Studenten beseffen dan:

Een branch is geen kopie van één bestand maar een alternatieve versie van het project.

5 Merge zonder conflict

Nu een eenvoudige merge.

main
voeg lijn toe:
Lieveingskleur
branch
voeg lijn toe:
Geboorteplaats
Ander deel van hetzelfde bestand.
Merge.
Resultaat:
beide lijnen aanwezig.
Zo leren ze:
Git kan veel merges automatisch oplossen.

6 Merge met conflict

Dit is een natuurlijk vervolg.

main Woonplaats: Gent
branch Woonplaats: Kortrijk
Merge.
Nu ontstaat een conflict.
Git weet niet welke versie juist is.
VSCode toont:
Current Incoming Both
Laat studenten expliciet kiezen.
Daarna commit.
Dit is waarschijnlijk het belangrijkste hoofdstuk van de hele cursus.
Veel beginners denken dat Git magie is.
Een conflict laat zien dat Git soms gewoon hulp nodig heeft.

7 Samenwerken met meerdere personen

Nu pas meerdere gebruikers.

Anders wordt het cognitief te zwaar.
Verhaal
Alice, Bob en Charlie werken aan een groepswerk.
Repository:
projectgroep
Bestanden:
inleiding.txt resultaten.txt besluit.txt
Alice
werkt aan inleiding.
Bob

werkt aan resultaten.
Charlie
werkt aan besluit.
Iedereen:
pull wijzigen commit push
Hier ontstaan meestal geen conflicten.
Dat toont waarom Git nuttig is.

8 Conflicten tussen groepsleden

Nu geef je doelbewust overlap.
Iedereen past dezelfde lijn aan.
Bijvoorbeeld:
Titel project
Studenten moeten:
pull conflict zien bespreken beste versie kiezen commit uitvoeren
Dit is eigenlijk de eerste echte samenwerkingssituatie.

9 Issues en feature requests

Pas nadat studenten begrijpen wat samenwerking is.
Anders lijkt een issue een vreemd extraatje.
Nieuwe terminologie issue bug feature request assignee Oefening
Laat repository bevatten:
studentenapp
Voorbeelden:
Bug Naam wordt niet opgeslagen.
Feature request Voeg donkere modus toe.
Verbetering Voeg export naar PDF toe.
Studenten maken zelf enkele issues.
Daarna:
issue toewijzen; sluiten na oplossing.
Zo ontstaat het verband:
issue ↓ branch ↓ commit ↓ merge ↓ issue sluiten

10 To-do's en projectbeheer

Pas helemaal op het einde.
Nu kunnen GitHub Projects of eenvoudige task boards geïntroduceerd worden.
Kolommen:
To do Doing Done
Voorbeeld:
To do Inleiding schrijven Grafieken toevoegen Besluit nalezen

Studenten slepen taken doorheen de workflow.
Samenvattende leerlijn
Ik zou de terminologie ongeveer in deze volgorde introduceren:
repository commit push pull branch merge merge conflict collaborator issue
feature request project board
Dat komt overeen met de natuurlijke vragen die studenten krijgen:
Hoe bewaar ik een versie?
→ commit
Hoe zet ik die online?
→ push
Hoe haal ik wijzigingen terug?
→ pull
Hoe experimenteer ik veilig?
→ branch
Hoe combineer ik werk?
→ merge
Wat als we hetzelfde aanpassen?
→ conflict
Hoe organiseren we teamwork?
→ issues en project boards
Zo blijft elk hoofdstuk even concreet als je huidige eerste deel en introduceer je telkens slechts één of twee nieuwe concepten tegelijk. Dat voorkomt dat Git voor beginners een verzameling losse commando's wordt.